

Definition

→ Push Down Automata, PDA, are a way to represent the language class called Context Free Languages(CFL).

→ The machine of the DFA is augmented by adding a stack, which we called PDA.

→ The additional parameters τ and z , the stack alphabet and the start stack symbol z are added to the five parameters making its a septuple.

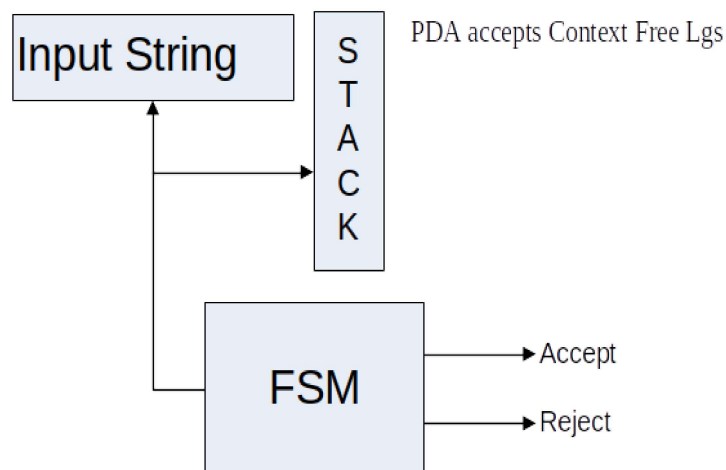


Fig : a PDA

Mathematically:

$$M = (Q, \Sigma, \tau, \delta, q_0, z_0, F)$$

where,

Q = a finite set of states with q_0 .

Σ = a finite alphabet of input symbols
(on the input tape).

τ = a finite set of push down stack
symbols including z_0 .

δ = a group of nondeterministic
transitions mapping.

$(Q \times (\Sigma \cup \{\epsilon\}) \times \tau$ to finite sets of $Q \times \tau^*$)

q_0 = the initial state, an element of Q

z_0 = the initial stack contents.

F = the set of final accepting states
but could be empty for a PDA.

"accepting on an empty stack"

Transition Rules :

Unlike finite automata, the δ is not presented in tabular form. The table would be too wide. δ is a list of, nondeterministic, transitions of the form:

$$\delta(q, a, A) = \{ (q_i, \tau_i), (q_j, \tau_j), \dots \}$$

where,

$q \rightarrow$ is the current state,

$a \rightarrow$ is the input tape symbol being read, an element of Σ union $\{\epsilon\}$

$A \rightarrow$ s the top of the stack being read.

A transition is given by

$$\delta(q_i, a, z) = (q_j, x)$$

Since, exactly when the push operation or the pop operation is to be started is not known for Non-deterministic PDA.

Lemma:

Every finite automaton can be viewed as a pushdown automaton that never operates on its stack.

Proof : Let $M = (Q, \Sigma, \delta, q_0, F)$ be a finite automaton and let $M' = (Q, \Sigma, \tau, \delta, q_0, z_0, F)$.

Put $\tau = \phi$ (i.e. no stack alphabet) then,

$$\delta' = \{((p, u, e), (q, e)) : (p, u, q) \in \delta\}$$

Then, M and M' accept the same language. i.e. FA can be viewed as PDA without stack operation.

Example

Design a PDA M to accept the language $L = \{wcw^R : w \in \{a, b\}^*\}$.

For example, $ababcbaba \in L$, but $abcbab \notin L$ and $cbc \notin L$.

We let $M = (Q, \Sigma, \tau, \delta, q_0, z_0, F)$, where $Q = \{s, f\}$, $\Sigma = \{a, b, c\}$, $\tau = \{a, b\}$, and $F = \{f\}$,

And δ contains the following five transitions.

1. $((s, a, e), (s, a))$
2. $((s, b, e), (s, b))$
3. $((s, c, e), (f, e))$
4. $((f, a, a), (f, e))$
5. $((f, b, b), (f, e))$
6. $((s, b, a), (s, ba))$
7. $((s, b, b), (s, bb))$
8. $((s, a, a), (s, aa))$
9. $((s, a, b), (s, ab))$

The sequence of Transitions :

State	Unread Input	Stack	Transition Used
s	abbcbbba	e	–
s	bbcbba	a	1
s	bcbbba	ba	6
s	cbba	bba	7
f	bba	bba	3
f	ba	ba	5
f	a	a	5
f	e	e	4

Accepted.

PDA & Context Free Grammar

Lemma : Each context-free language is accepted by some pushdown automaton.

Proof : Let $G = (V, T, P, S)$ be a CFG; we must construct a pushdown automaton M such that $L(M) = L(G)$. The machine we construct has only two states, p & q , and remains permanently in state q after its first move. Also, M uses V , the set of terminals and non-terminals, as its stack alphabet. We let $M = (\{p, q\}, T, V, \delta, p, z_0, \{q\})$, where δ contains the following transitions.

1. $((p, e, e), (q, S))$
2. $((q, e, A), (q, x))$ for each rule $A \rightarrow x$ in P .
3. $((q, a, a), (q, e))$ for each $a \in T$.

The pushdown automaton M begins by pushing S , the start symbol of G , on its

initially empty pushdown store, and entering state q . On each subsequent step, it either replaces the topmost symbol A on the stack, provided that it is a non-terminal, by the right-hand side x of some rule $A \rightarrow x$ in P , or pops the topmost symbol from the stack, provided that it is a terminal symbol that matches the next input symbol. The transitions of M are designed so that the pushdown store during an accepting computation mimics a leftmost derivation of the input string: M intermittently carries out a step of such a derivation on the stack, and between such steps it strips away from the top of the stack any terminal symbols matches them against symbols in the input string.

Examples

1. Consider the grammar $G = (V, T, P, S)$ with,

$$V = \{S, a, b, c\}$$

$$T = \{a, b, c\}$$

$$P = \{ S \rightarrow aSa, S \rightarrow bSb, S \rightarrow c \}$$

which generates the language $\{wcw^R : w \in \{a,b\}^*\}$.

The PDA for above is $M = (\{p, q\}, T, V, \delta, p, z_0, \{q\})$ with,

$$\delta = \{$$

1. $((p, e, e), (q, S))$,
2. $((p, e, S), (q, aSa))$,
3. $((p, e, S), (q, bSb))$,
4. $((p, a, a), (p, e))$,
5. $((p, b, b), (p, e))$,
6. $((p, e, S), (q, c))$,
7. $((q, a, a), (q, e))$,
8. $((q, b, b), (q, e))$,
9. $((q, c, c), (q, e))$

$\}$.

The string abbcbbba is accepted by M through the following sequence of moves.

<i>State</i>	<i>Unread Input</i>	<i>Stack</i>	<i>Transition Used</i>
p	abbcbbba	e	–
p	abbcbbba	S	1
p	abbcbbba	aSa	2
p	bbcbba	Sa	4
p	bbcbba	bSba	3
p	bcbbba	Sba	5
p	bcbbba	bSbba	3
p	cbba	Sbba	5
q	cbba	cbba	9
q	bba	bba	8
q	ba	ba	8
q	a	a	7
q	e	e	–

2. PDA accepts an Arithmetic Expression

The grammar $G = (V, T, P, S)$ that is given for arithmetic expression is,

$$V = \{E, E', T, T', F, id\}$$

$$T = \{+, -, *, /, id, (,)\}$$

$$S = \{E\}$$

The production P is given :

$$E \rightarrow TE',$$

$$E' \rightarrow +TE' \mid \varepsilon,$$

$$T \rightarrow FT',$$

$$T' \rightarrow *FT' \mid \varepsilon,$$

$$F \rightarrow (E) \mid id$$

The PDA M for above is : $M = (\{p, q\}, T, V, \delta, p, z_0, q)$ with,

$$\delta = \{$$

$$1. ((p, e, e), (q, E)),$$

$$2. ((q, e, E), (q, TE')),$$

$$3. ((q, e, T), (q, FT')),$$

$$4. ((q, (, F), (q, (E))),$$

$$5. ((q, (, (, (q, e))),$$

6. $((q, id, F), (q, id)),$
 7. $((q, id, id), (q, e)),$
 8. $((q, e, T'), (q, e)),$
 9. $((q, +, E'), (q, +TE')),$
 10. $((q, +, +), (q, e)),$
 11. $((q, e, E'), (q, e)),$
 12. $((q,),), (q, e)),$
 13. $((q, *, T'), (q, *FT')),$
 14. $((q, *, *), (q, e)),$
- }.

The string $(id+id)*(id*id)$ is accepted by M through the following sequence of moves.

State	Unread Input	Stack	Transition Used
p	$(id+id)*(id*id)$	e	–
q	$(id+id)*(id*id)$	E	1
q	$(id+id)*(id*id)$	TE'	2
q	$(id+id)*(id*id)$	FT'E'	3
q	$(id+id)*(id*id)$	(E)T'E'	4
q	$id+id)*(id*id)$	E)T'E'	5
q	$id+id)*(id*id)$	TE')T'E'	2
q	$id+id)*(id*id)$	FT'E')T'E'	3
q	$id+id)*(id*id)$	idT'E')T'E'	6

q	+id)*(id*id)	T'E')T'E'	7
q	+id)*(id*id)	E')T'E'	8
q	+id)*(id*id)	+TE')T'E'	9
q	id)*(id*id)	TE')T'E'	10
q	id)*(id*id)	FT'E')T'E'	3
q	id)*(id*id)	idT'E')T'E'	6
q	)*(id*id)	T'E')T'E'	7
q	)*(id*id)	E')T'E'	8
q	)*(id*id)	)T'E'	11
q	*(id*id)	T'E'	12
q	*(id*id)	*FT'E'	13
q	(id*id)	FT'E'	14
q	(id*id)	(E)T'E'	4
q	id*id)	E)T'E'	5
q	id*id)	TE')T'E'	2
q	id*id)	FT'E')T'E'	3
q	id*id)	idT'E')T'E'	6
q	*id)	T'E')T'E'	7
q	*id)	*FT'E')T'E'	13
q	id)	FT'E')T'E'	14
q	id)	idT'E')T'E'	6
q	)	T'E')T'E'	7
	)	E')T'E'	8
	)	)T'E'	11
	e	T'E'	12
	e	E'	8
	e	e	11

3. Infix to Postfix conversion with PDA

A PDA can also be used to convert infix expression with a postfix one.

Infix : $a + b * c$

Postfix : $a b c * +$

The procedure is given below :

First we need to prepare a precedence table. The precedence table for above expression is given below;

\$	→ 0
+ or -	→ 1
* or /	→ 2
id	→ 3 (highest priority)

When the input symbol is having higher precedence than the stack symbol then the input symbol is put on the stack. If it is equal or less, then stack top symbols popped of and given as output.

– (p, a + b * c \$, \$)	
– (p, + b * c \$, a \$)	
– (p, + b * c \$, \$)	o/p : a
– (p, b * c \$, + \$)	o/p : a
– (p, * c \$, b + \$)	o/p : a
– (p, * c \$, + \$)	o/p : a b
– (p, c \$, * + \$)	o/p : a b
– (p, \$, c * + \$)	o/p : a b
– (p, \$, * + \$)	o/p : a b c
– (p, \$, + \$)	o/p : a b c *
– (p, \$, \$)	o/p : a b c * +

The final output is $a b c * +$.

The Pumping Lemma for CFL's

We show that if certain strings belong to a language, then certain other strings must also belong to the language.

If L is a CFL, then strings of L that are at least m symbols long can be pumped to produce additional strings in L . The value of m depends on the particular language.

Theorem

Let L be an infinite CFL. Then there is some positive integer m such that, if S is a string of L of length at least m , then

- (a) $S = uvwxy$ [For some u, v, w, x and y]
- (b) $|vwx| \leq m$
- (c) $|vx| \geq 1$
- (d) $uv^iwx^iy \in L$, For all non-negative values of i .

Consideration, It should be understood that,

(a) If S is sufficiently long string, then there are two sub-strings, v and x , somewhere in S . There is stuff (u) before v , stuff (w) between v and x , and stuff (y), after x .

(b) The stuff between v and x won't be too long, because $|vwx|$ can't be larger than m .

(c) Sub-strings v and x won't both be empty, though either one could be.

(d) If we duplicate sub-string v , some number (i) of times, and duplicate x the same number of times, the resultant string will also be in L .

Proof of Pumping Lemma:

Let us consider a string X belonging to L . [L is infinite, L does not contain λ , G (of L) has no λ -production]

If X is sufficiently long, then the derivation of X must have involved recursive use of some variable A .

Since A was used derivation, the derivation should have started as

$$s \xrightarrow{*} uAy$$

for some values of u and y . Since A was used recursively the derivation must have continued as

$$s \xrightarrow{*} uAy \xrightarrow{*} uvAxy$$

Finally the derivation must have eliminated all variables to reach a string X in the language.

$$s \xrightarrow{*} uAy \xrightarrow{*} uvAxy \xrightarrow{*} uvwxy = X$$

This shows that derivation steps $A^* \rightarrow vAx$ and $A^* \rightarrow w$ are possible. Hence the derivation $A^* \rightarrow vwx$ must also be possible.

It should be noted here that the above does not imply that A was used recursively only once. The $*$ of $^* \rightarrow$ could cover many uses of A , as well as other recursive variables.

There has to be some last recursive step. Consider the longest string that can be derived for v , w and x without use of recursion. Then there is a number m such that $|vwx| < m$. Since the grammar does not contain any ϵ -productions or unit productions, every derivation step either introduces a terminal or increases the length of the sentential form. Since, $A^* \rightarrow vAx$, It follows that $|vx| \geq 1$.

Finally, since $uvAxy$ occurs in the derivation, and $A^* \rightarrow vAx$ and $A^* \rightarrow w$ are both possible, it follows that uv^iwx^iy also belongs to L .

This completes the proof of all parts of Lemma.

Usage of Pumping Lemma

The pumping Lemma can be used to show that certain languages are not context free.

Example

Let us show that the language $L = \{a^i b^i c^i \mid i > 0\}$ is not a context free.

Proof :

Suppose L is a CFL. If string $X \in L$, where $|X| > m$, it follows that $X = uvwxy$, where $|vwx| \leq m$.

Choose a value i that is greater than m . Then, wherever vwx occurs in the string $a^i b^i c^i$, it cannot contain more than two distinct letters. It can be all a 's, all b 's, all c 's or it can be a 's and b 's, or it can be b 's and c 's.

Therefore the string vx cannot contain more than two distinct letters; but by

the pumping lemma, it cannot be empty either, so it must contain at least one letter i.e. $|vx| \geq 1$.

Now we are ready to pump. Since $uvwxy$ is in L , uv^2wx^2y must also be in L . Since v and x can't both be empty.

$$|uv^2wx^2y| > |uvwxy|$$

So, we have added letters. Both since vx does not contain all three distinct letters, we cannot have added the same number of each letter.

Therefore, uv^2wx^2y cannot be in L . Thus we have arrived at a contradiction. Hence our assumption, that L is context free should be false. Hence the language L is not context free.

Properties of CFL

Lemma : The context free language is closed under union, concatenation and star-closure.

Proof :

Let L_1 and L_2 are two CFLs defined by the following respective CFGs.

$$\begin{aligned} G_1 &= (V_1, T_1, P_1, S_1) \quad \text{and} \\ G_2 &= (V_2, T_2, P_2, S_2) \end{aligned}$$

Union: Let $L(G)$ be the language generated by the grammar $G = (V, T, P, S)$ where,

$$\begin{aligned} V &= V_1 \cup V_2 \cup \{S\}, \\ T &= T_1 \cup T_2 \quad \text{and} \\ P &= \{P_1 \cup P_2 \cup \{S \rightarrow S_1 \mid S_2\}\}. \end{aligned}$$

Now, If we select a string $\omega \in \{V_1UV_2\}^*$ then, $S_1 \xrightarrow{*} \omega$ or $S_2 \xrightarrow{*} \omega$ and also we have $S \rightarrow S_1 \mid S_2$. Therefore, S will derive ω .

Hence, G is a CFG and $L(G)$ is a CFL i.e. $L(G) = L_1 \cup L_2$ is a CFL.

Concatenation: Let $L(G)$ be the language generated by the grammar $G = (V, T, P, S)$ where,

$$V = V_1 \cup V_2 \cup \{S\},$$

$$T = T_1 \cup T_2 \text{ and}$$

$$P = \{P_1 \cup P_2 \cup \{S \rightarrow S_1 S_2\}\}.$$

Now, If we select a string $\omega_1 \in L_1$ and $\omega_2 \in L_2$ then, $S_1 \xrightarrow{*} \omega_1$ and $S_2 \xrightarrow{*} \omega_2$ also in grammar G , we have $S \rightarrow S_1 S_2$. Therefore, S will derive w i.e. $\omega_1 \omega_2$. This shows that the grammar G derives $L_1 L_2$. We know that L_1 and L_2 are CFLs and G is CFG which leads $L_1 L_2$ is also a CFL.

Star-closure : Let L be a CFL generated by a CFG $G = (V, T, P, S)$. Now, let the Star-closure of a language L i.e. L^* is generated by grammar $G_1 = (V_1, T_1, P_1, S_1)$ where, $V_1 = V$, $T_1 = T$ and the production P_1 is given by,

$$P_1 = \{P \cup \{S_1 \rightarrow S, S_1 \rightarrow \epsilon, S_1 \rightarrow SS\}\}.$$

So, Star-closure of a language L i.e. L^* can be generated by G_1 which is a CFG and hence L^* is also a CFL.

Example:

Prove that $L = \{\omega\omega \mid \omega \in \{a,b\}^*\}$ is not a CFL.

Suppose L is a CFL and a constant k can be obtained from the pumping lemma i.e.

$$\omega = a^k b^k a^k b^k \in L.$$

Using pumping lemma, we can decompose w of string length m in 5 substrings i.e.

$$\omega = uvwxy \text{ where,}$$

$|vx| \geq 1$ and $|vwx| \leq m$ then lemma allows us that $uv^iwx^iy \in L$ for all $i \geq 1$.

We have $|vwx| \leq m$ i.e. vwx contains only a 's, say. Again, we know that $|vx| \geq 1$ that means either v or x contains at least one a .

For $i = 2$ gives $uvvwxxy \in L$. This will increase only a 's but not b 's. So, $uv^2wx^2y \notin L$. This leads to contradiction and proves that L is not regular.

Decision Algorithm for CFL

The followings are the decision question on CFG:

1. Can we identify whether two different CFG's defines the same language ?
2. How do we identify the given CFG is ambiguous or not ?
3. Can we convert the ambiguous CFG to its equivalent non-ambiguous CFG ?
4. Can we identify whether two different CFG's have a common word (sentence) ?
5. How can we identify the set of words that is not generated by a given CFG ?
6. Emptiness: Given a CFG, can it generates any words ?
 - a. Remove all ϵ -productions and replace all NT by terminals according to their productions.

- b. Stop (a) to eliminate the starting NT by its generated string.

Example :

$S \rightarrow AB, A \rightarrow aC, B \rightarrow bb, C \rightarrow c$

Using (a),

$S \rightarrow Abb, A \rightarrow ac, S \rightarrow acbb$

Using(b), Stop.

This generates at least one word.

7. Finiteness: How to decide whether a CFG generates finite language or infinite language ?

→ Check whether any productions have repeating NT (variable), that helps to identify whether it generates infinite or finite language.

8. Membership: Can we identify whether a given word w over alphabet S be generated by a CFG ?

→ Convert CFG to CNF and try to generate all possible sub-string of w by using all available production rules.